Modern rendering patterns

7B. Modern Rendering Patterns

Questions: 4 **Difficulty:** Senior **Companies:** Google, Amazon, Flipkart, Razorpay, Swiggy, Microsoft

---

Q 7.5 What is Streaming SSR? How does it improve the user experience?

Difficulty: Senior **Asked at:** Google, Amazon, Flipkart, Razorpay

Traditional SSR has a problem: the server must finish rendering the entire page before sending any HTML. If the page depends on a slow database query or API call, the user waits for everything — even the parts of the page that don't depend on that slow data.

Streaming SSR sends HTML to the browser in chunks as it's rendered. The server starts flushing HTML immediately — the header, navigation, and layout shell arrive first. When a slow data source responds, that section's HTML is sent and seamlessly replaces a placeholder.

Traditional SSR vs Streaming SSR:

JAVASCRIPT Copy

```
Traditional SSR:
  ──────────────────

  Server: [──────── fetch ALL data ────────][── render HTML ──] → send complet
  Browser: [──────────── waiting ──────────────────────────────] → display pag
```

```
Streaming SSR:
──────────────

Server: [render shell] → flush → [fetch slow data] → [render section] →
Browser: [shell visible] → [shows skeleton] → [content streams in]

User sees the page shell in 200ms, content fills in progressively.
```

How it works with React Suspense:

JSX Copy

```jsx
import { Suspense } from 'react';

function ProductPage({ params }) {
  return (
    <main>
      <Header />  {/* Sent immediately */}

      <Suspense fallback={<ProductSkeleton />}>
        <ProductDetails slug={params.slug} />
        {/* This component fetches data. While waiting,
            the skeleton is sent. When data arrives,
            the real content streams in and replaces it. */}
      </Suspense>

      <Suspense fallback={<ReviewsSkeleton />}>
        <ProductReviews slug={params.slug} />
        {/* Independent Suspense boundary — reviews can
            stream in separately from product details.
            If reviews API is slow, product details
            aren't held up. */}
      </Suspense>

      <Footer />  {/* Sent immediately */}
    </main>
  );
}
```

`ProductDetails` finishes fetching, the server sends a `<script>` tag that swaps the skeleton with the real content. Same for reviews. Each section is independent — a slow review service doesn't delay product details.

What the browser receives (simplified):

HTML Copy

```
<!-- Chunk 1: Sent immediately -->
<header>...</header>
<div id="product-placeholder">
  <div class="skeleton">Loading product...</div>
</div>
<div id="reviews-placeholder">
  <div class="skeleton">Loading reviews...</div>
</div>
<footer>...</footer>

<!-- Chunk 2: Sent when product data arrives (maybe 300ms later) -->
<script>
  // Replaces the product skeleton with actual content
  document.getElementById('product-placeholder').innerHTML =
    '<h1>Nike Air Max</h1><p>₹12,995</p>...';
</script>

<!-- Chunk 3: Sent when reviews arrive (maybe 800ms later) -->
<script>
  // Replaces the reviews skeleton with actual content
  document.getElementById('reviews-placeholder').innerHTML =
    '<div class="reviews">...</div>';
</script>
```

Why this matters:

- TTFB (Time to First Byte) drops dramatically — server starts flushing within milliseconds
- FCP (First Contentful Paint) improves — user sees the shell immediately
- Perceived performance improves — skeleton screens create the illusion of speed

- Slow APIs don't block the entire page — each Suspense boundary is independent

Follow-up: "Does streaming work with CDN caching?"

Partially. You can cache the shell at the CDN and stream the dynamic parts from the server. But most CDNs expect a complete response before caching, so fully dynamic streaming isn't CDN-cacheable. The pattern is usually: static shell from CDN + streamed dynamic sections from the origin server.

Q 7.6 What are React Server Components (RSC)? How do they change the server/client boundary?

Difficulty: Senior **Asked at:** Google, Amazon, Flipkart, Razorpay, Swiggy

React Server Components are components that run exclusively on the server. They never ship to the browser. Their JavaScript is never downloaded by the client. They can directly access databases, file systems, and internal services — things that normally require an API layer.

The fundamental shift:

Before RSC, every React component was a client component — its code was bundled and sent to the browser, even if it only rendered static content. A product listing component that just displayed data from a database still shipped to the client as JavaScript, added to the bundle size, and hydrated on the client.

With RSC, you split components into two types:

Server Components (default in Next.js App Router):

- Run on the server only
- Access databases, file systems, environment variables directly
- Their code is never sent to the browser (zero bundle impact)
- Can't use state, effects, or browser APIs (no useState, no useEffect, no onClick)
- Can render other Server Components or Client Components

- Can use state, effects, event handlers — everything you're used to
- Their code IS sent to the browser (adds to bundle size)
- Can only render other Client Components (can receive Server Components as children via props)

JSX Copy

```jsx
// Server Component (default) - runs ONLY on server
// This component's code is never sent to the browser
async function ProductPage({ params }) {
  // Direct database access - no API route needed
  const product = await db.product.findUnique({
    where: { slug: params.slug },
  });

  return (
    <main>
      <h1>{product.name}</h1>
      <p>{product.description}</p>
      <ProductImages images={product.images} />  {/* Server Component */}

      {/* Client Component - needs interactivity */}
      <AddToCartButton productId={product.id} price={product.price} />
    </main>
  );
}
```

JSX Copy

```jsx
// Client Component - needs onClick handler
'use client';

import { useState } from 'react';

function AddToCartButton({ productId, price }) {
  const [added, setAdded] = useState(false);
```

```
  }

  return (
    <button onClick={handleClick} disabled={added}>
      {added ? 'Added to Cart' : `Add to Cart — ₹${price}`}
    </button>
  );
}
```

In this example, `ProductPage` and `ProductImages` are Server Components — their code is never sent to the browser. Only `AddToCartButton` is a Client Component that ships to the client. If `ProductPage` is 500 lines of rendering logic, none of that adds to the bundle.

When to use Server vs Client Components:

Use Server Components for: data fetching, database access, rendering static or semi-static content, accessing backend services, heavy dependencies (markdown rendering, syntax highlighting — these libraries stay on the server).

Use Client Components for: interactivity (click handlers, form inputs), browser APIs (localStorage, geolocation), state management (useState, useReducer), effects (useEffect, subscriptions), third-party libraries that use browser APIs.

The mental model: Start with everything as Server Components (the default). Add `"use client"` only when you need interactivity. Push the `"use client"` boundary as deep as possible — instead of making an entire page a Client Component because the header has a theme toggle, make only the theme toggle a Client Component.

Follow-up: "Can Server Components pass data to Client Components?"

Yes, through props — but only serializable data (strings, numbers, arrays, plain objects). You can't pass functions, classes, or Dates. This constraint exists because the server serializes the props to send them to the client. If you need a function, define it inside the Client Component.

Q 77 What is Progressive Hydration? How does it improve interactivity?

Frontend System Desi...

Difficulty: Senior **Asked at:** Google, Amazon, Microsoft

Standard hydration is all-or-nothing — React hydrates the entire page at once. If your page has 50 components, all 50 are hydrated before anything is interactive. The JavaScript for a footer component is parsed and executed before the user can click the "Add to Cart" button at the top of the page.

Progressive hydration breaks hydration into priorities. Critical components (above-the-fold content, primary CTA buttons) hydrate first. Below-the-fold components, sidebars, and footers hydrate later — on idle, on visibility, or on interaction.

Why standard hydration is problematic:

A typical e-commerce product page has: header navigation, product images carousel, product details, add-to-cart button, related products grid, 50 reviews, a FAQ accordion, and a footer. Standard hydration processes ALL of these before the "Add to Cart" button works. The user sees the button (server-rendered HTML), clicks it, and nothing happens because React hasn't hydrated it yet.

How Progressive Hydration prioritizes:

JSX Copy

```jsx
import { Suspense, lazy } from 'react';

function ProductPage() {
  return (
    <main>
      {/* Hydrate immediately — critical for interaction */}
      <ProductDetails />
      <AddToCartButton />

      {/* Hydrate when visible in viewport */}
      <Suspense fallback={<div>Loading reviews...</div>}>
        <LazyReviews />
      </Suspense>

      {/* Hydrate on idle — not urgent */}
```

```
      {/* Hydrate on interaction — only when user engages */}
      <Suspense fallback={<div>FAQ</div>}>
        <LazyFAQ />
      </Suspense>
    </main>
  );
}
```

React's `Suspense` with streaming SSR achieves a form of progressive hydration — each Suspense boundary hydrates independently. `React.lazy()` splits the code so less-critical components download and hydrate only when needed.

Hydration strategies by priority:

- **Immediate:** Above-the-fold content, primary CTAs, navigation — hydrate as soon as JS loads
- **On visibility:** Below-the-fold sections — hydrate when the user scrolls to them (IntersectionObserver)
- **On idle:** Non-critical widgets — hydrate during `requestIdleCallback` when the main thread is free
- **On interaction:** Components that need JS only when engaged — hydrate on first click, hover, or focus (e.g., an accordion that only needs JS when someone clicks it)

Follow-up: "How does this relate to React 18's concurrent features?"

`useTransition` and `useDeferredValue` in React 18 let you mark hydration work as non-urgent. The browser can interrupt hydration to handle user input (clicks, typing), then resume hydration afterward. This means even during hydration, the page stays responsive to user interactions. Combined with Suspense boundaries, this creates a naturally progressive hydration experience without manually implementing visibility observers.

Q 7.8 What is Islands Architecture? When does it make sense?

Difficulty: Senior **Asked at:** Google, Razorpay, Flipkart

Islands Architecture treats a page as mostly static HTML with isolated "islands" of interactivity. Instead of hydrating the entire page (like React SSR), only the interactive islands are hydrated. The rest of the page is pure, zero-JavaScript HTML.

The concept:

Imagine a blog post page. 95% of it is static content — the article text, headings, images, author bio, footer. Only a few elements are interactive — the comment form, the share button, the table of contents with scroll tracking. In a traditional SPA, the entire page is a React application. All the static content is still part of the React component tree, still hydrated, and still costs JavaScript.

With Islands Architecture, the static content is pure HTML. Only the interactive pieces are "islands" — small, independent JavaScript widgets embedded in the static page:

HTML Copy

```
<!-- Static HTML - no JavaScript, no hydration -->
<article>
  <h1>Understanding Closures in JavaScript</h1>
  <p>Closures are one of the most misunderstood concepts ... </p>
  <!-- 2000 words of content - all static HTML -->
</article>

<!-- Island: Interactive table of contents (small JS widget) -->
<toc-island>
  <nav data-island="TableOfContents">
    <!-- This small component hydrates independently -->
  </nav>
</toc-island>

<!-- Island: Comment section (needs React for form handling) -->
<comments-island>
  <section data-island="Comments">
    <!-- This component hydrates independently -->
  </section>
</comments-island>
```

```
  <p>Published by Ankit on March 15, 2026</p>
</footer>
```

Astro is the primary framework implementing this pattern:

ASTRO Copy

```
---
// This runs at build time (server-only)
const post = await getPost(Astro.params.slug);
---

<!— Static HTML — zero JavaScript shipped —>
<article>
  <h1>{post.title}</h1>
  <div set:html={post.content} />
</article>

<!— Interactive island — React component, hydrated client-side —>
<TableOfContents client:visible headings={post.headings} />

<!— Another island — only hydrates when user scrolls to it —>
<Comments client:visible postId={post.id} />

<!— Another island — only hydrates on user interaction (click) —>
<ShareButtons client:idle url={post.url} />

<footer>
  <p>Published {post.date}</p>
</footer>
```

Astro's `client:` directives control when each island hydrates:

- `client:load` — hydrate immediately when the page loads
- `client:idle` — hydrate when the browser is idle (requestIdleCallback)
- `client:visible` — hydrate when the island enters the viewport
- `client:media="(max-width: 768px)"` — hydrate only on mobile
- No directive — renders static HTML only, zero JavaScript

When Islands Architecture makes sense:

- Content-heavy sites (blogs, documentation, news) where 90%+ of the page is static
- Marketing sites where performance is critical and interactivity is minimal
- Sites where different islands can use different frameworks (a React comment widget + a Svelte chart + vanilla JS share buttons)

When it doesn't make sense:

- Highly interactive applications (dashboards, email clients, design tools) where most of the page is interactive
- Applications with complex shared state between components (if islands need to talk to each other frequently, the architecture fights you)
- Teams deeply invested in a single framework (Next.js, Remix) that handles rendering well already

The numbers that matter:

A typical blog page built as a React SPA might ship 150-300KB of JavaScript. The same page built with Astro's Islands Architecture might ship 10-30KB (only the JavaScript for the interactive widgets). This difference is transformative for mobile users on slow networks.

Follow-up: "How does this compare to React Server Components?"

RSC and Islands solve overlapping problems differently. RSC keeps everything in the React ecosystem — Server Components don't ship JS, Client Components do. The boundary is at the component level within React. Islands Architecture is framework-agnostic — the static shell can be plain HTML or any templating language, and islands

page.

QUICK CHECK — no marks, just for you.

1. A product page streams with SSR and wraps ProductDetails and ProductReviews in two separate Suspense boundaries. The reviews API takes 800ms while the product API takes 300ms. What does the user experience?

- ○ Both sections appear together at ~800ms, because sibling Suspense boundaries resolve as a group
- ○ The reviews boundary throws and is dropped from the response because it exceeded the product boundary's timing
- ○ Nothing renders until both APIs resolve, since the server must flush the page as one complete response
- ○ The product details appear at ~300ms while a reviews skeleton is still showing, and reviews fill in later

Check answer

2. A Server Component fetches an order from the database and wants to hand data down to a Client Component. Which prop will NOT work?

- ○ onFormat, a function the Client Component should call to format the total
- ○ items, an array of plain objects describing each line item
- ○ total, a number holding the order amount
- ○ status, a string such as "shipped"

Check answer

3. How do React 18's useTransition and useDeferredValue help while a large page is still hydrating?

- They shrink the client bundle by keeping the deferred components' JavaScript on the server
- They let hydration work be marked non-urgent, so the browser can pause it to handle a click or keystroke and resume afterward
- They move hydration off the main thread into a Web Worker so it never competes with user input
- They skip hydration entirely for components that are below the fold until the user scrolls to them

Check answer

Previous
Core rendering strategies

Next
Rendering and loading

LEARN MORE

ARTICLES & DOCS

- Rendering on the web (web.dev)
  web.dev
- Patterns.dev
  patterns.dev

VIDEOS



React Server Components vs SSR
YouTube



Frontend System Desi...



Streaming SSR can fix a slow app

YouTube

Hand-picked references for this lesson.